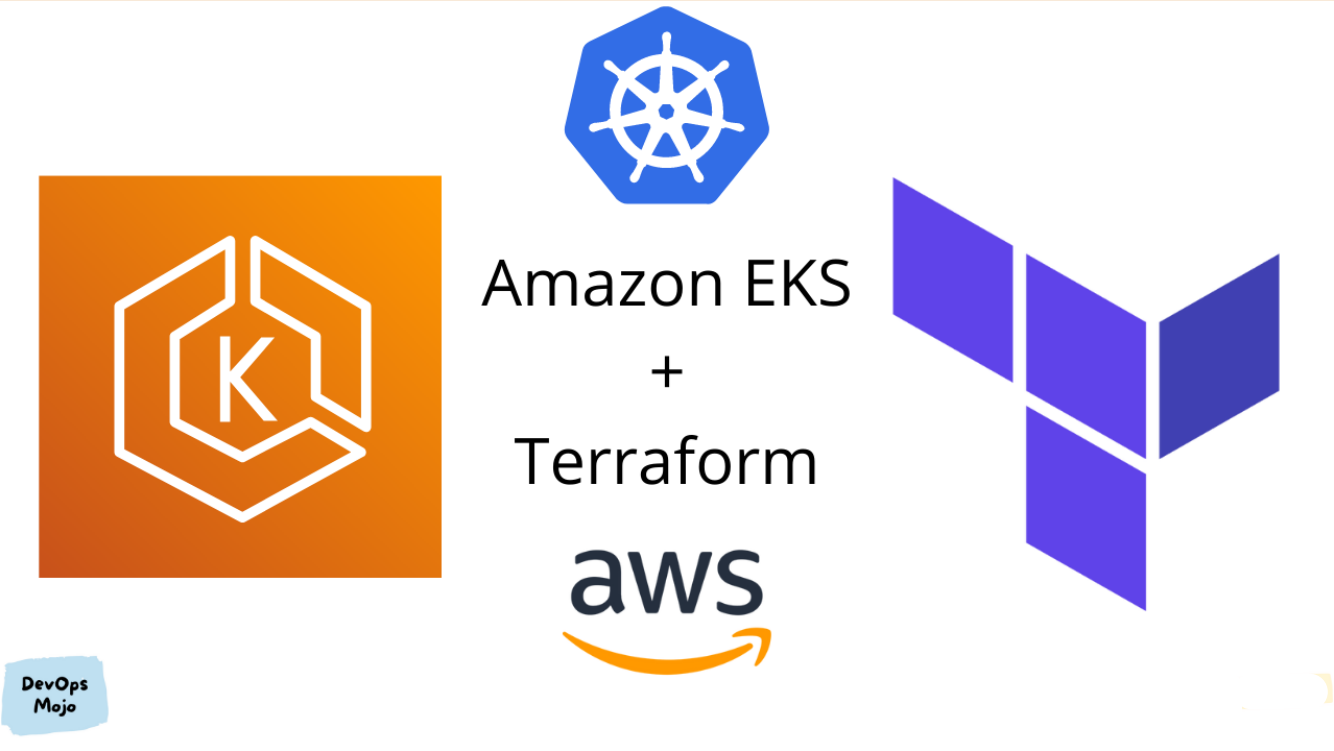
Shashank Padhye

Deploying an Amazon EKS Cluster using Terraform



INTRODUCTION TO TERRAFORM

Terraform is an open-source infrastructure as code tool created by HashiCorp. It enables you to define and provision data center infrastructure using a high-level configuration language known as HCL (HashiCorp Configuration Language).

- Declarative language for defining infrastructure
- Supports a wide range of cloud providers, including AWS
- Provides a consistent CLI workflow to manage hundreds of nodes and services in diverse environments

STACK FLOW

INFRASTRUCTURE AS CODE

Terraform

- HCL (HashiCorp Configuration Language)
- Terraform AWS Provider
- Terraform Modules
  - terraform-aws-modules/eks/aws

CLOUD PROVIDER

- AWS (Amazon Web Services)
  - IAM (Identity and Access Management)
    - IAM User with Administrator Access
    - Access Key and Secret Access Key
  - AWS CLI (Command Line Interface)
  - EKS (Elastic Kubernetes Service)
  - VPC (Virtual Private Cloud
  - Route Tables

KUBERNETES MANAGEMENT

- kubectl (Kubernetes Command Line Tool)
  - ConfigMap/aws/auth

OPERATING SYSTEM

Windows

- Command Prompt/ GIT Bash

USE CASES OF TERRAFORM

MULTI-CLOUD SUPPORT:

Write infrastructure code that can deploy to multiple cloud providers

CHANGE MANAGEMENT:

Terraform makes it easy to automate changes and updates to your infrastructure

DISASTER RECOVERY

Terraform can help you quickly recover from outages by re-deploying your infrastructure

INFRASTRUCTURE AS CODE:

Define infrastructure in a file that can be versioned, reused, and shared

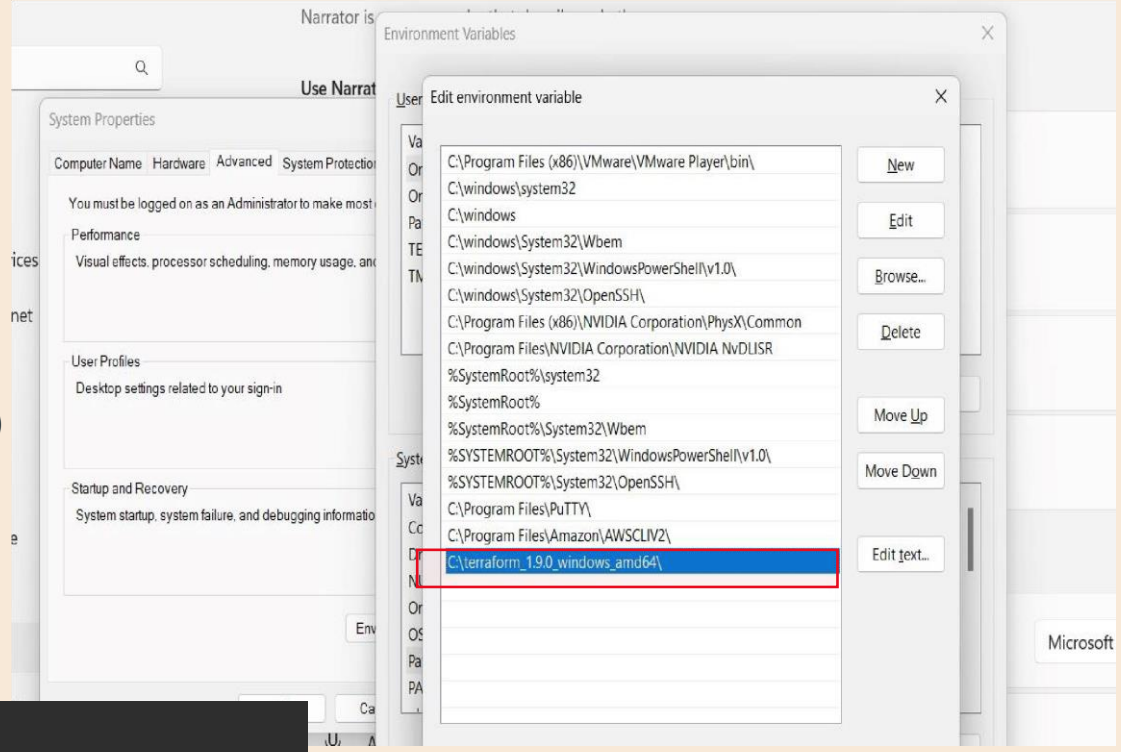
STEP1: INSTALLATION OF TERRAFORM

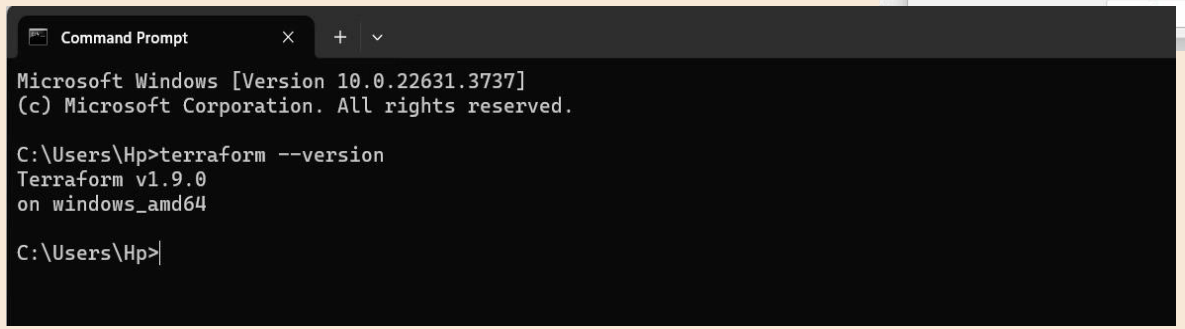
1.Download Terraform: -

- https://developer.hashicorp.com/terraform/install Downloads.
- Download the 64bit version for Windows as per the system configuration.

2.Install Terraform: -

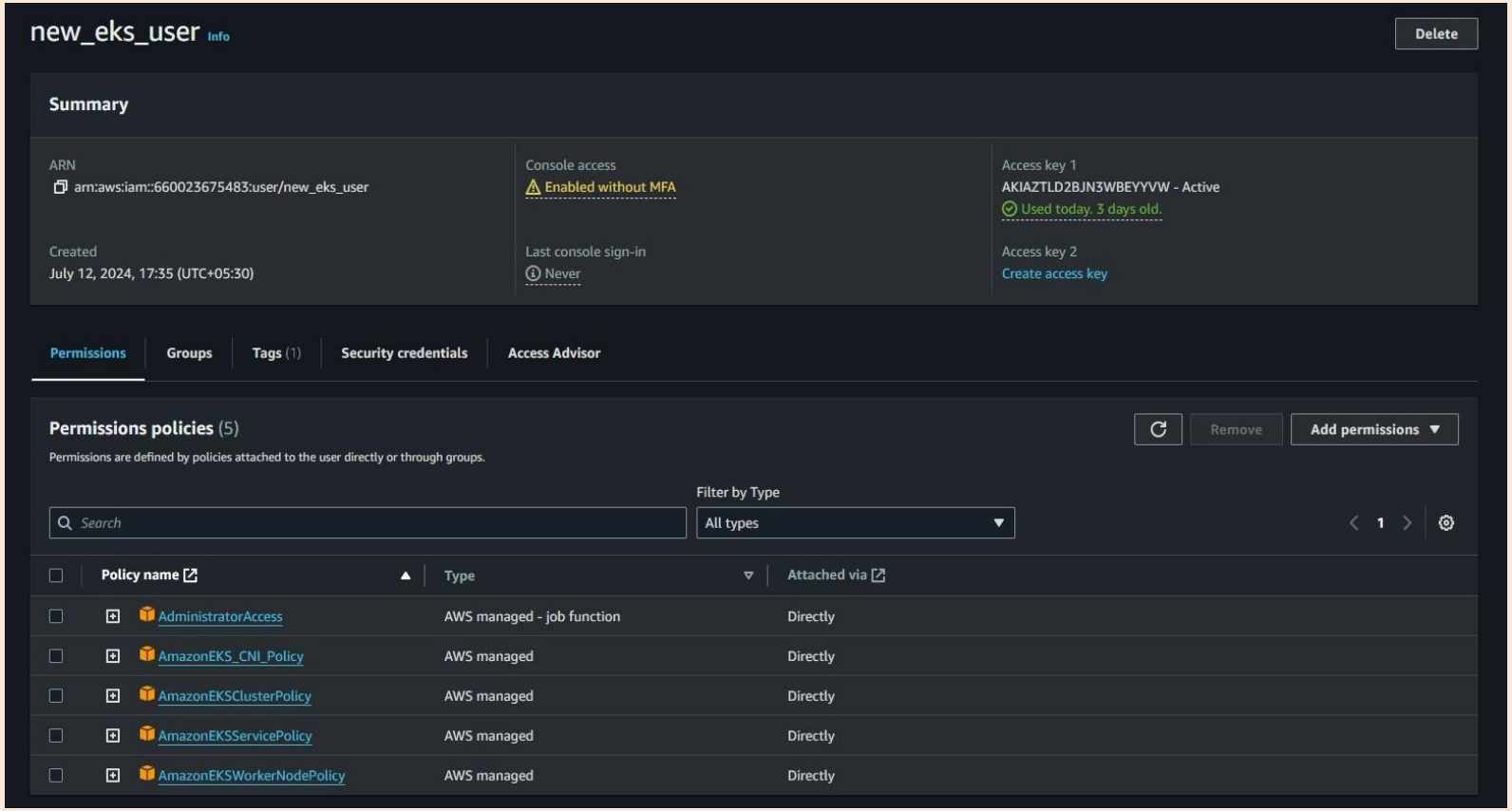
- Extract the downloaded zip file.
- Move the terraform.exe file to a directory included in your
- system's PATH. You can add a new directory (e.g., C:\Terraform)
- to your PATH if it’s not already included:
- Go to settings and search for system environment variables
- Click on Advanced system settings.
- Click the Environment Variables button.
- Add the path to the directory containing terraform.exe (e.g., C:\Terraform).





STEP2:CREATING IAM USER

Create Access key and Secret Access key by navigating to security credentials after creating the users



STEP3: AWS CONFIGURE

Prerequisite to configure AWS: Install AWS CLI on you local device
https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\last_attempt_eks> aws configure
AWS Access Key ID [****************YYVW]:
AWS Secret Access Key [****************fUcp]:
Default region name [us-east-1]:
Default output format [json]:
PS C:\last_attempt_eks> aws sts get-caller-identity
{
    "UserId": "AIDAZTLD2BJN3HY7MVS3P",
    "Account": "660023675483",
    "Arn": "arn:aws:iam::660023675483:user/new_eks_user"
}

PS C:\last_attempt_eks>
```

STEP4: CREATING TERRAFORM CONFIGURATION

- Create a custom directory for the terraform configuration files
- Create a new directory (e.g., "C:\last_attempt_eks")
- Create the required terraform configuration files under the created directory

1) Main.tf file: This file includes the Terraform modules to create an EKS cluster using the module “terraform-awsmodules/eks/aws” and the configuration to create a cluster.

File Edit Selection View Go Run Terminal Help

last_attempt_eks

EXPLORER

LAST_ATTEMPT_EKS

- .terraform
  - modules
  - providers
- .terraform.lock.hcl
- kubectl.exe.sha256
- main.tf
- provider.tf
- terraform.plan
- terraform.tfstate
- terraform.tfstate.backup
- tfplan
- vpc.tf

main.tf | vpc.tf | provider.tf

main.tf > module "eks" > eks_managed_node_groups > sp-cluster-wg

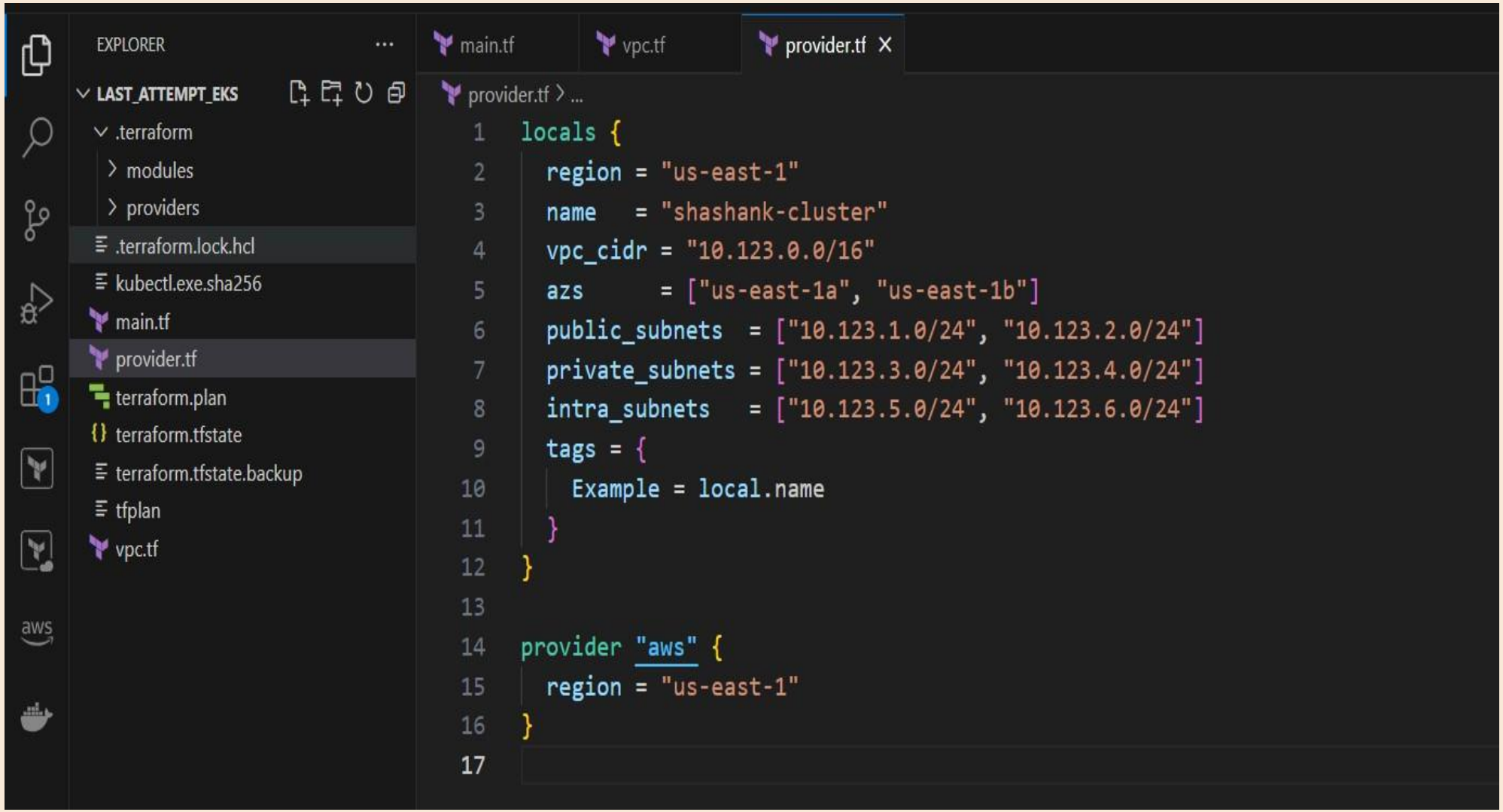
```tf
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "19.15.1"

  cluster_name                   = local.name
  cluster_endpoint_public_access = true

  cluster_addons = {
    coredns = {
      most_recent = true
    }
    kube-proxy = {
      most_recent = true
    }
    vpc-cni = {
      most_recent = true
    }
  }

  vpc_id                   = module.vpc.vpc_id
  subnet_ids               = module.vpc.private_subnets
  control_plane_subnet_ids = module.vpc.intra_subnets

  eks_managed_node_group_defaults = {
    ami_type       = "AL2_x86_64"
    instance_types = ["m5.large"]

    attach_cluster_primary_security_group = true
  }

  eks_managed_node_groups = {
    sp-cluster-wg = {
      min_size     = 1
      max_size     = 2
      desired_size = 1

      instance_types = ["t3.large"]
      capacity_type  = "SPOT"

      tags = {
        ExtraTag = "helloworld"
      }
    }
```

kubectl.exe.sha256
main.tf
provider.tf
terraform.plan
terraform.tfstate
terraform.tfstate.backup
tfplan
vpc.tf

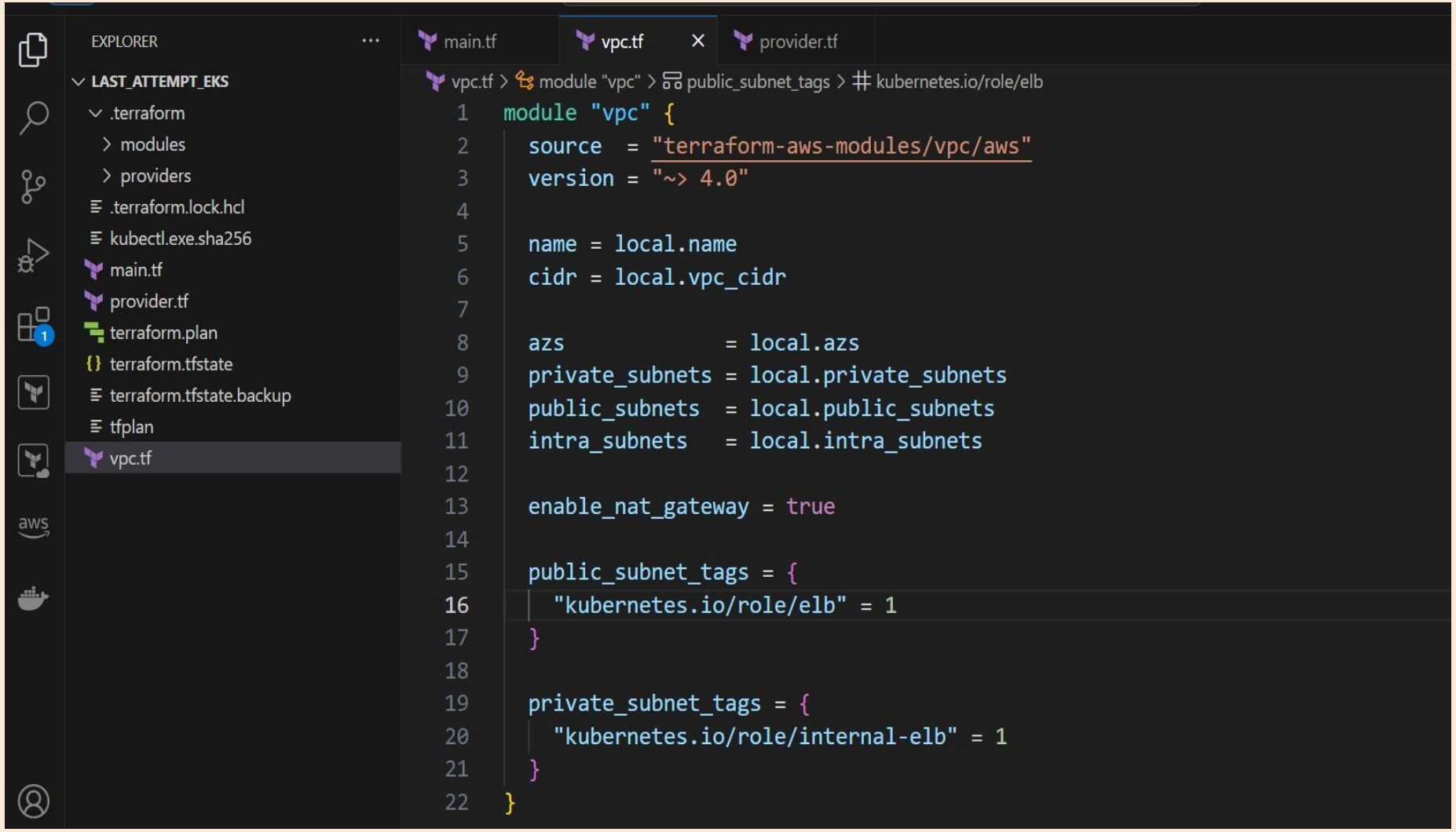
```
        instance_types = ["t3.large"]
        capacity_type  = "SPOT"

        tags = {
          ExtraTag = "helloworld"
        }
      }
    }

    tags = local.tags
  }

  output "eks_cluster_id" {
    value = module.eks.cluster_id
  }

  output "eks_cluster_endpoint" {
    value = module.eks.cluster_endpoint
  }

  output "eks_cluster_security_group_id" {
    value = module.eks.cluster_security_group_id
  }

  output "eks_node_security_group_id" {
    value = module.eks.node_security_group_id
  }

  output "vpc_id" {
    value = module.vpc.vpc_id
  }

  output "private_subnets" {
    value = module.vpc.private_subnets
  }

  output "public_subnets" {
    value = module.vpc.public_subnets
  }

```

OUTLINE
TIMELINE

Providers.tf

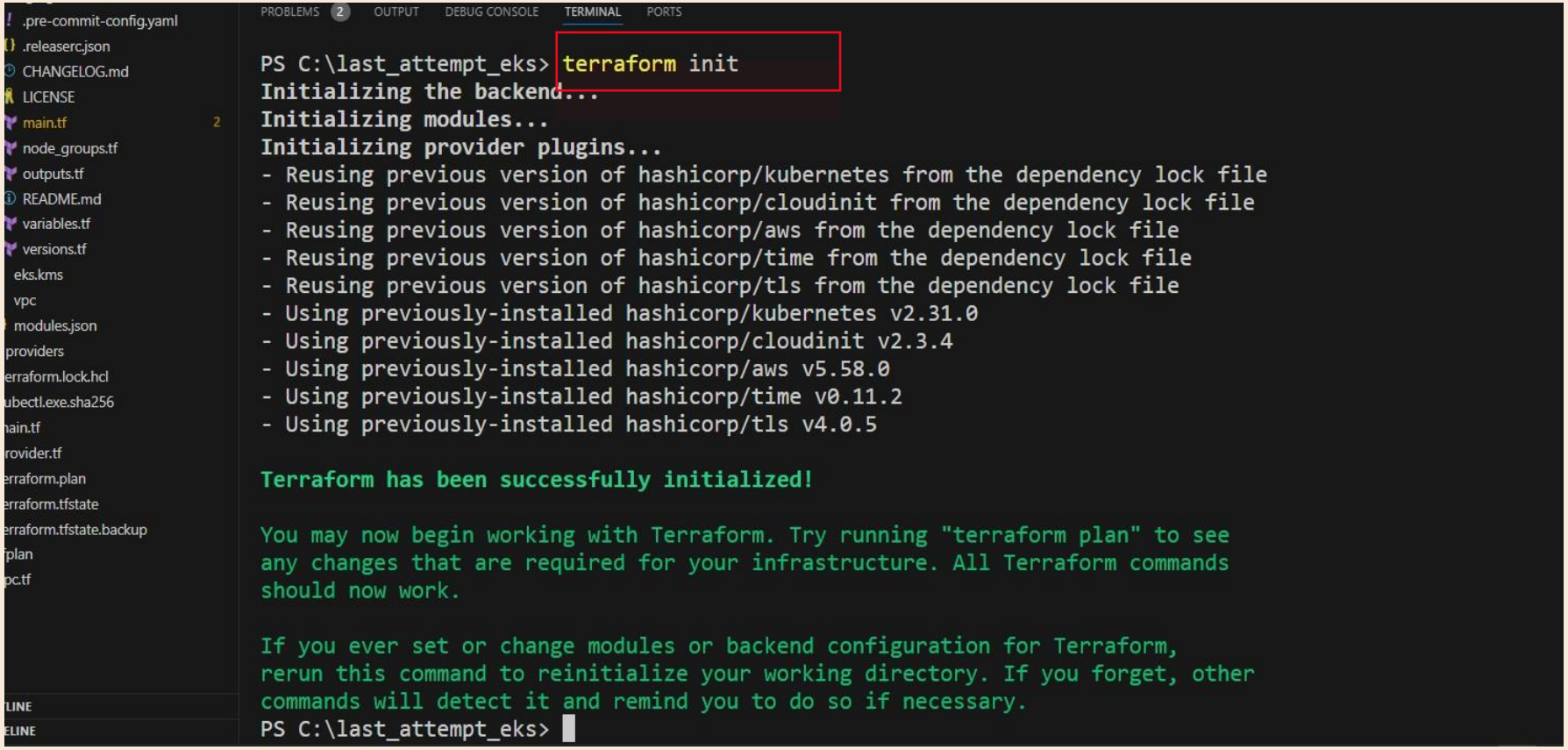


vpc.tf


EXPLORER
LAST_ATTEMPT_EKS
.terraform
modules
providers
.terraform.lock.hcl
kubectl.exe.sha256
main.tf
provider.tf
terraform.plan
terraform.tfstate
terraform.tfstate.backup
tfplan
vpc.tf
main.tf
vpc.tf
provider.tf
vpc.tf > module "vpc" > public_subnet_tags > kubernetes.io/role/elb
module "vpc" {
source = "terraform-aws-modules/vpc/aws"
version = "~> 4.0"
name = local.name
cidr = local.vpc_cidr
azs = local.azs
private_subnets = local.private_subnets
public_subnets = local.public_subnets
intra_subnets = local.intra_subnets
enable_nat_gateway = true
public_subnet_tags = {
"kubernetes.io/role/elb" = 1
}
private_subnet_tags = {
"kubernetes.io/role/internal-elb" = 1
}
}

STEP5:INITIALIZING THE TERRAFORM

- Navigate to your project directory in Command Prompt.
- Run “terraform init” to initialize the project.
- This will download the necessary provider plugins and modules.

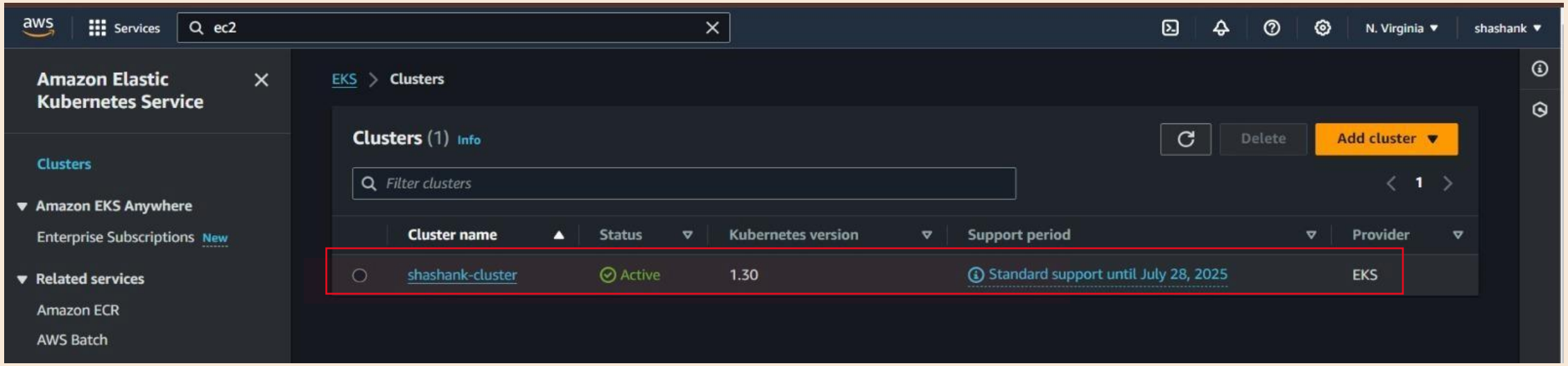


STEP6: APPLY THE CONFIGURATION FOR TERRAFORM

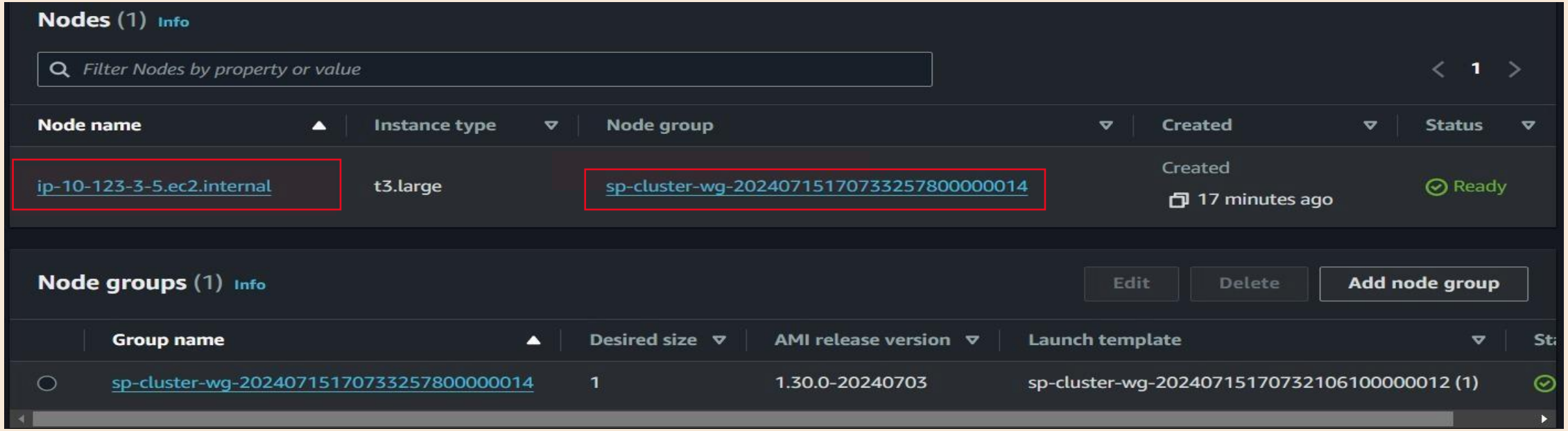
Run the terraform apply command to execute the configuration file. This will create the VPC, subnets, route tables, and other infrastructure needed to create an EKS cluster and launch the EKS cluster in the desired region.

```
module.eks.aws_eks_addon.this["kube-proxy"]: Creating...
module.eks.aws_eks_addon.this["coredns"]: Creating...
module.eks.aws_eks_addon.this["vpc-cni"]: Creating...
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [10s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [10s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [10s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Creation complete after 10s [id=amonkincloud-cluster:kube-proxy]
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [20s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [20s elapsed]
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [30s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [30s elapsed]
module.eks.aws_eks_addon.this["vpc-cni"]: Still creating... [40s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [40s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Creation complete after 48s [id=amonkincloud-cluster:coredns]
module.eks.aws_eks_addon.this["vpc-cni"]: Creation complete after 49s [id=amonkincloud-cluster:vpc-cni]

Warning: Argument is deprecated

  with module.eks.aws_eks_addon.this["kube-proxy"],
  on .terraform\modules\eks\main.tf line 392, in resource "aws_eks_addon" "this":
 392:   resolve_conflicts        = try(each.value.resolve_conflicts, "OVERWRITE")

The "resolve_conflicts" attribute can't be set to "PRESERVE" on initial
resource creation. Use "resolve_conflicts_on_create" and/or
"resolve_conflicts_on_update" instead

(and 4 more similar warnings elsewhere)

Apply complete! Resources: 62 added, 0 changed, 0 destroyed.

Outputs:

eks_cluster_endpoint = "https://02ED52FCA73D95F14C226EAA8441A31E.gr7.us-east-1.eks.amazonaws.com"
eks_cluster_security_group_id = "sg-063bc0210146c66a7"
eks_node_security_group_id = "sg-04eb4a770420b29a7"
private_subnets = [
  "subnet-0d108a2b0245b86f4",
  "subnet-076e99a6e5e9cf22d",
]
public_subnets = [
  "subnet-058b6a14008562667",
  "subnet-0f8b850fa6a423901",
]
```

STEP7: VERIFICATION OF NEWLEY CREATED RESOURCES



Cluster nodes and group



Cluster info Info

f

Overview | Resources | Compute | Networking | Add-ons | Access | Observability | Upgrade insights | Update history | Tags

Networking

Manage VPC resources | Manage endpoint access

VPC Info
vpc-0c875aac528d5b97f

Cluster IP address family Info
IPv4

Service IPv4 range Info
172.20.0.0/16

Subnets
subnet-0eaa2e050567dbeb4
subnet-077f19afebf78f7b3

Cluster security group Info
sg-076cb44ea5bebc6ff

Additional security groups
sg-0f2a98ce51f4725bd

API server endpoint access Info
Public and private

Public access source allowlist
0.0.0.0/0 (open to all traffic)

---

aws | Services | Search [Alt+S] | N. Virginia | shashank

EC2 Dashboard
EC2 Global View
Events
Console-to-Code Preview
Instances

Instances (1) Info

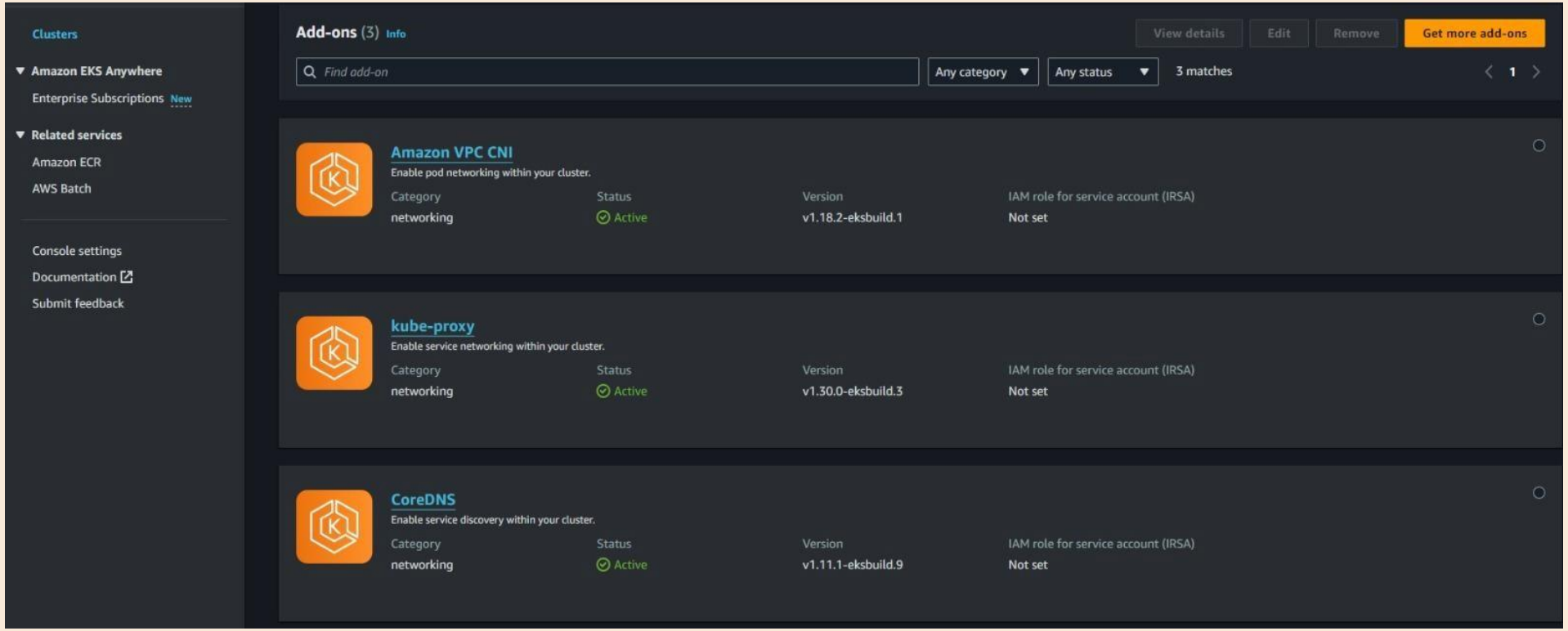
Connect | Instance state | Actions | Launch instances

Find Instance by attribute or tag (case-sensitive) | All states | 1

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DN
sp-cluster-wg	i-0d1ef4dde59bc52c9	Running	t3.large	2/2 checks passed	View alarms +	us-east-1a	–

Addons for Cluster reliability

1. Amazon VPC CNI ensures efficient and secure pod networking by integrating directly with AWS VPC, providing each pod with an IP address from the VPC's CIDR block.
2. Kube-proxy maintains network rules to facilitate service discovery and load balancing, ensuring traffic is routed correctly within the cluster.
3. CoreDNS provides DNS services for the cluster, enabling service discovery and allowing services and pods to communicate using DNS names.



Prerequisite: Installation of “kubectl” on Windows

Reference: https://docs.aws.amazon.com/eks/latest/userguide/install-kubectl.html

```
Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/windows/amd64/kubectl.exe"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   138  100   138    0     0    220      0 --:--:-- --:--:-- --:--:--   221
100 50.3M  100 50.3M    0     0  2012k      0  0:00:25  0:00:25 --:--:-- 2205k

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/windows/amd64/kubectl.exe.sha256"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   138  100   138    0     0    193      0 --:--:-- --:--:-- --:--:--   194
100    64  100    64    0     0     41      0  0:00:01  0:00:01 --:--:--   125

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$
^C

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ echo "$(cat kubectl.exe.sha256)  kubectl.exe" | sha256sum --check
kubectl.exe: OK

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ mkdir -p /c/kubectl

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ mv kubectl.exe /c/kubectl/kubectl.exe

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ echo 'export PATH=$PATH:/c/kubectl' >> ~/.bashrc
source ~/.bashrc

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe version --client
Client Version: v1.30.2
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
```

connect to the cluster using the Command Prompt / windows PowerShell to create pods on the cluster.

```
PS C:\last_attempt_eks> aws eks update-kubeconfig --region us-east-1 --name shashank-cluster
Added new context arn:aws:eks:us-east-1:660023675483:cluster/shashank-cluster to C:\Users\Hp\.kube\config
PS C:\last_attempt_eks>
```

After successful connection edit the ConfigMAp/aws/auth in the “kubectl.yml” file which opens up using the following command

```
Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe edit configmap aws-auth -n kube-system
configmap/aws-auth edited

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe run --image=ngnix ngnix
pod/ngnix created

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ |
```

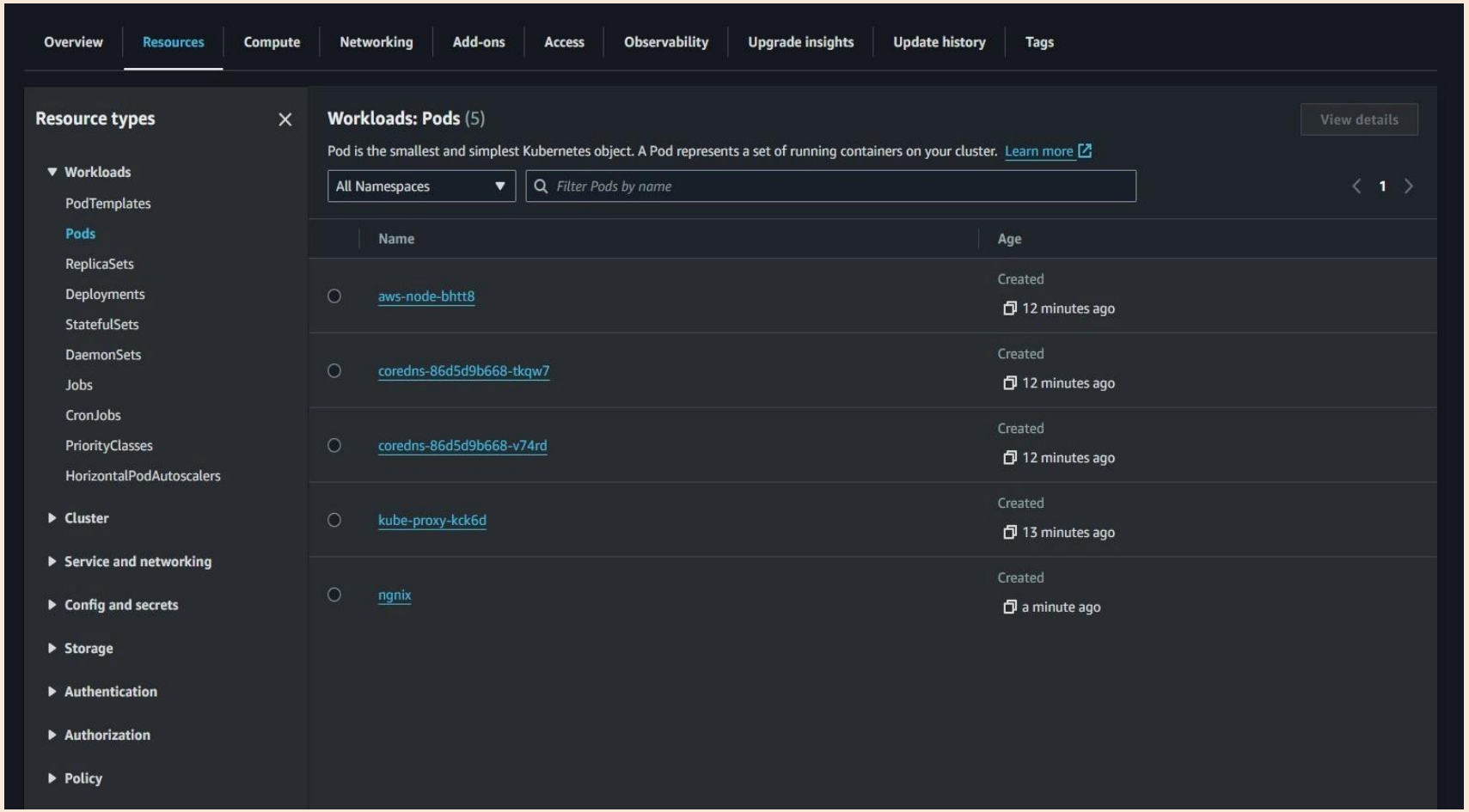
Enter the rolearn of the node group and mapUser to root

```
apiVersion: v1
data:
  mapRoles: |
    - groups:
      - system:bootstrappers
      - system:nodes
      rolearn: arn:aws:iam::660023675483:role/sp-cluster-wg-eks-node-group-20240715165806752500000001
      username: system:node:{{EC2PrivateDNSName}}
  mapUsers: |
    - userarn: arn:aws:iam::660023675483:root
      username: root
      groups:
        - system:masters
kind: ConfigMap
metadata:
  creationTimestamp: "2024-07-15T17:08:15Z"
  name: aws-auth
  namespace: kube-system
  resourceVersion: "1061"
  uid: 3ae230b1-ee10-47aa-8b18-7e701cddbe83
```

Now we can crate pods on our node group

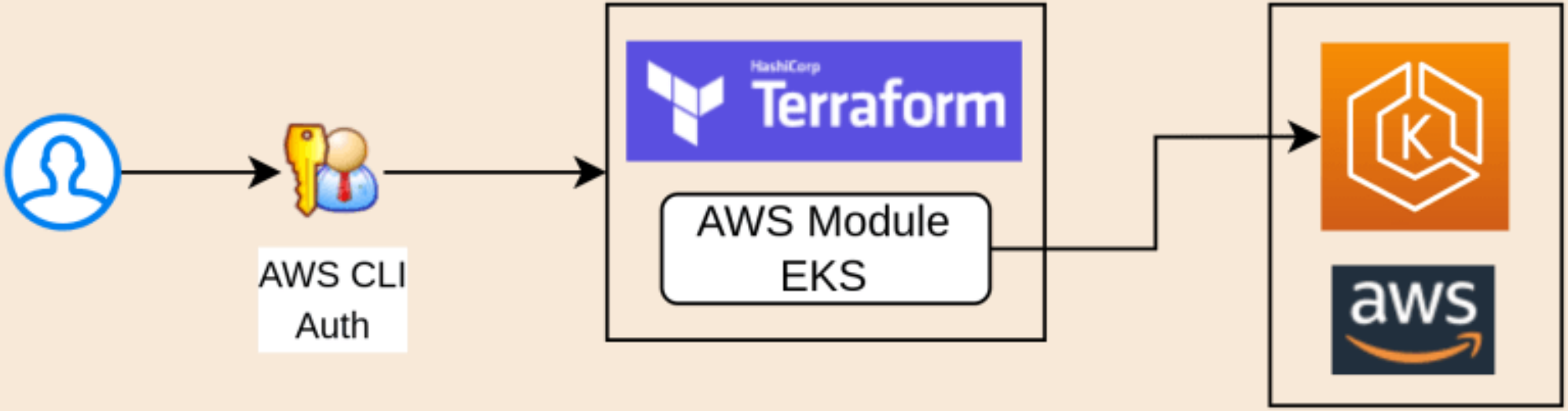
```
Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe edit configmap aws-auth -n kube-system
configmap/aws-auth edited

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe run --image=ngnix ngnix
pod/ngnix created

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ |
```

Verify the newly created pod



```
Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe get nodes
NAME                          STATUS   ROLES    AGE   VERSION
ip-10-123-3-5.ec2.internal    Ready    <none>   12m   v1.30.0-eks-036c24b

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ kubectl.exe get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   172.20.0.1   <none>        443/TCP   21m

Hp@LAPTOP-H5JRHJ5I MINGW64 /c/last_attempt_eks
$ |
```

REFERENCES

1. https://registry.terraform.io/providers/hashicorp/aws/latest/docs/resources/eks_cluster
2. https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html
3. https://docs.aws.amazon.com/eks/latest/userguide/install-kubectl.html
4. https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli

Thank You